
Desarrollo de un sistema empujado para
procesamiento de una red neuronal
Embedded system design for accelerating a
neural network



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Andrés Saumell Caminos

Director

Hortensia Mecha López

Juan Carlos Fabero

Grado en Ingeniería de Computadores

Facultad de Informática

Universidad Complutense de Madrid

Desarrollo de un sistema empotrado para
procesamiento de una red neuronal
Embedded system design for accelerating a
neural network

Trabajo de Fin de Grado en Ingeniería de Computadores

Autor

Andrés Saumell Caminos

Director

Hortensia Mecha López

Juan Carlos Fabero

Convocatoria: *Junio* 2026

Grado en Ingeniería de Computadores

Facultad de Informática

Universidad Complutense de Madrid

12 de mayo de 2026

Dedicatoria

*A mi madre, por enseñarme que a pesar de
todo, hay que seguir adelante.*

Resumen

Desarrollo de un sistema empotrado para procesamiento de una red neuronal

En este trabajo se estudia, diseña y se implementa un sistema empotrado heterogéneo para acelerar procesos de Inteligencia Artificial, comúnmente llamado Edge-AI. Este desarrollo se lleva a cabo utilizando la plataforma Digilent Zybo, la cual integra un procesador ARM Cortex-A9 con una FPGA Xilinx/AMD Artix-7, llevando la ejecución de la inferencia de la red neuronal a dicha FPGA y poder maximizar el cómputo total. El proyecto sigue una metodología de codiseño hardware-software empleando herramientas de Síntesis de Alto Nivel (HLS) y buses de comunicación AXI4 con controladores DMA para el movimiento eficiente de datos.

El código puede consultarse en:

<https://github.com/andysaull/zybo-heterogeneous-cifar10-inference>

Palabras clave

Sistema empotrado, Inteligencia Artificial, Edge AI, FPGA, Síntesis de Alto Nivel.

Abstract

Embedded system design for accelerating a neural network

This project studies, designs and implements a heterogeneous embedded system to accelerate Artificial Intelligence processes, commonly referred to as Edge-AI. This development is performed using the Digilent Zybo platform, which integrates an ARM Cortex-A9 processor with a Xilinx/AMD Artix-7 FPGA, offloading the execution of neural network inference to the FPGA to maximize overall computational performance. The project follows a hardware-software co-design methodology using High-Level Synthesis (HLS) tools and AXI4 communication buses with DMA controllers for efficient data transfer.

The code is available at:

<https://github.com/andysaull/zybo-heterogeneous-cifar10-inference>

Keywords

Embedded system, Artificial Intelligence, Edge AI, FPGA, High Level Synthesis.

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	2
2. Estado de la Cuestión	5
2.1. Arquitectura Zynq-7000 y Comunicaciones AXI	5
2.2. Síntesis de Alto Nivel (HLS)	6
2.3. hls4ml y generación de modelos para FPGA	7
2.4. IA en FPGAs con recursos limitados	7
2.5. Herramientas utilizadas en el flujo	8
2.6. Panorama industrial y proyectos de referencia	8
2.6.1. Microsoft Project Catapult	8
2.6.2. NVDLA	9
2.6.3. BNN-PYNQ-ZYBO	9
3. Descripción del Trabajo	11
3.1. Configuración del entorno y preparación (Fase 1)	11
3.2. Comunicación PS-PL (Fase 2)	12
3.3. Desarrollo del acelerador base en HLS: multiplicación de matrices (Fase 3)	12
3.3.1. Diseño del acelerador mediante Síntesis de Alto Nivel (HLS) .	13
3.3.2. Integración a Nivel de Sistema y Arquitectura de Memoria . .	13
3.3.3. Implementación software y coherencia de caché	13
3.3.4. Evaluación de rendimiento (Benchmarking)	14
3.4. Diseño e implementación del modelo de IA (Fase 4)	14
3.4.1. Entrenamiento y selección del modelo	15
3.4.2. Conversión del modelo a HLS	16
3.4.3. Adaptación del IP core para AXI DMA	16
3.4.4. Integración en Vivado	17
3.4.5. Aplicación bare-metal en Vitis	18
3.4.6. Interpretación de la salida	18

3.4.7. Problemas encontrados durante la integración	19
3.5. Pruebas y benchmarking (Fase 5)	19
3.5.1. Preparación de pruebas de recursos	19
3.5.2. Generación de imágenes de prueba en formato Q16.16	20
3.5.3. Aplicaciones de inferencia y medición	21
3.5.4. Criterios de interpretación	21
4. Resultados y evaluación	23
4.1. Comparación de recursos entre HLS y VHDL	23
4.2. Utilización de recursos del acelerador CIFAR-10	24
4.3. Validación funcional de la inferencia	25
4.4. Benchmark de inferencia	26
4.5. Valoración global	28
5. Conclusiones y trabajo futuro	29
5.1. Cumplimiento de objetivos	29
5.2. Lecciones aprendidas	30
5.3. Limitaciones	30
5.4. Trabajo futuro	31
Introduction	33
Motivation	33
Objectives	34
Work plan	34
Conclusions and Future Work	37
Achievement of the objectives	37
Lessons learned	38
Limitations	38
Future work	39
Bibliografía	41

Índice de figuras

3.1. Diseño de bloques para la aceleración CNN de CIFAR-10	17
3.2. Imagen reducida con la herramienta <code>image_file_to_h.py</code>	20

Índice de tablas

3.1. Resultados en tiempo de la multiplicación de matrices paralelas . . .	14
4.1. Comparación de recursos entre la multiplicación de matrices en VHDL y en HLS	23
4.2. Utilización de recursos del diseño CIFAR-10 final en la Zybo	24
4.3. Resultados de inferencia para 10 imágenes CIFAR-10 mostrando la confianza por clase	25
4.4. Benchmark de inferencia CIFAR-10 entre acelerador FPGA y core C++ de hls4ml en ARM	26
4.5. Comparación de salida entre la inferencia en FPGA y la ejecución hls4ml en ARM	27

Introducción

El mundo de la informática en general atraviesa una transición importante. Con la explosión de la Inteligencia Artificial, procesar datos en tiempo real ha dejado de ser un lujo para convertirse en una necesidad. Si bien el enfoque tradicional pasaba por delegar el cómputo de las redes neuronales en grandes centros de datos en la nube, hoy la tendencia apunta hacia el concepto de Edge-AI: llevar la inteligencia directamente al extremo, al dispositivo local. ¿Qué logramos con esto? Reducir de forma drástica la latencia, blindar la privacidad del usuario al eliminar el envío de información por la red y aliviar la carga de las redes de comunicación. (Zhou et al., 2019).

Aquí es donde entran en juego las FPGAs como pieza clave. A diferencia de las CPUs y GPUs tradicionales que procesan la información de forma secuencial o en paralelo pero de forma rígida, las FPGAs permiten diseñar arquitecturas de hardware como si fueran un lienzo en blanco, que optimizan el flujo de datos y el paralelismo. Esto es importante para superar barreras físicas actuales como el "Muro de Potencia" (Power Wall), donde seguir subiendo la frecuencia de los procesadores convencionales ha dejado de ser viable a nivel energético. (Hennessey y Patterson, 2019)

Por todo ello, este Trabajo de Fin de Grado se centra en explorar a fondo el co-diseño hardware-software. Para materializarlo utilizaremos la plataforma Digilent Zybo (Digilent, 2024), una placa que combina la flexibilidad de un procesador ARM Cortex-A9 para tareas de control con la potencia de una FPGA Xilinx/AMD de la familia 7-series para tareas de cálculo intensivo en un mismo chip. A nivel académico es un entorno ideal para emular sistemas industriales, dándole al diseñador el poder de controlar cada ciclo de reloj y cada transferencia de datos a través de buses de alto rendimiento como AXI4.

1.1. Motivación

La demanda de capacidad de cómputo, impulsada principalmente por algoritmos de Inteligencia Artificial y Deep Learning, crece a un ritmo que los procesadores de propósito general ya no pueden satisfacer eficientemente bajo restricciones estrictas de consumo. Trasladar la inferencia desde la nube hasta los equipos locales no es

opcional, es el camino a seguir.

El verdadero reto técnico de este trabajo, y lo que lo hace tan desafiante, es intentar exprimir al máximo una FPGA con recursos modestos (apenas 80 bloques DSP) para hacer correr modelos de Machine Learning. Todo esto apoyándonos en herramientas modernas de Síntesis de Alto Nivel (HLS) para desmitificar y agilizar la histórica complejidad que siempre ha rodeado al diseño puramente hardware.

1.2. Objetivos

El objetivo general de este trabajo es claro: diseñar, implementar y evaluar un sistema empotrado heterogéneo basado en el SoC Zynq-7010 capaz de acelerar la inferencia de un modelo ligero de Inteligencia Artificial. Derivado a esto, se quiere cuantificar las mejoras de rendimiento en hardware respecto a la ejecución del mismo flujo de inferencia sobre el microprocesador ARM. Para alcanzar esta meta, se definen los siguientes objetivos específicos:

1. **Puesta en marcha de la plataforma:** Configurar desde cero el entorno de desarrollo Vivado/Vitis para la placa Digilent Zybo, validando el funcionamiento independiente del microprocesador ARM Cortex-A9 y de la FPGA.
2. **Diseño de la infraestructura de comunicación:** Implementar canales de comunicación de alto ancho de banda entre el Sistema de Procesamiento y la Lógica Programable, utilizando el protocolo AMBA AXI y controladores de Acceso Directo a Memoria (DMA).
3. **Aceleración de un cálculo base:** Escribir en C++ y sintetizar a hardware (HLS) un IP core básico de álgebra lineal (multiplicación de matrices) para validar el flujo de datos entre procesador y FPGA.
4. **Implementación de IA en recursos limitados:** Adaptar un modelo de red neuronal (Red Neuronal Convolutiva CIFAR-10) al conjunto reducido de recursos de la Zynq-7010, generando su hardware equivalente y desarrollando el software de control que le suministre los datos de entrada.
5. **Evaluación de rendimiento (Benchmarking):** Realizar una comparativa de tiempos de ejecución (latencia) lanzando la inferencia del modelo neuronal únicamente en la CPU frente a la ejecución asistida por la FPGA.

1.3. Plan de trabajo

Para organizar el esfuerzo y cumplir los hitos, el proyecto se divide en estas fases incrementales:

Fase 1. Investigación y Familiarización: Revisión del estado del arte en sistemas Zynq, buses AXI y Síntesis de Alto Nivel. Instalación de herramientas (Vivado, Vitis) y ejecución de "Hola Mundo." en la placa.

Fase 2. Comunicación PS-PL: Configuración de registros de control mediante AXI-Lite y configuración del subsistema de memoria con AXI DMA. Desarrollo de drivers en C (baremetal) para mover datos desde la RAM a la FPGA.

Fase 3. Desarrollo del Acelerador Base en HLS: Programación en C/C++ de una multiplicación de matrices. Uso de pragmas HLS (PIPELINE, UNROLL) para optimizar el hardware. Integración del bloque IP generado en Vivado con el DMA.

Fase 4. Diseño e implementación del modelo de IA: Entrenamiento y/o exportación de un modelo de Red Neuronal Convolucional para clasificación de imágenes CIFAR-10. Generación del acelerador hardware de la red neuronal y conexión en el diseño de Vivado.

Fase 5. Pruebas y Benchmarking: Ejecución de la inferencia en FPGA y comparación con el core generado por hls4ml ejecutado en el ARM. Toma de métricas de rendimiento y verificación de resultados.

Fase 6. Redacción de la Memoria: Documentación del proceso, análisis de los resultados obtenidos y redacción final del documento del TFG.

Estado de la Cuestión

Durante décadas, la evolución de la computación se apoyó en aumentar la frecuencia de los procesadores y en integrar más transistores en el mismo chip. Esa tendencia permitió que muchas aplicaciones mejorasen sin cambiar demasiado la forma de programarlas. Sin embargo, en los últimos años se ha hecho evidente que esa vía no escala igual de bien: el consumo, la disipación térmica y la dificultad de explotar más paralelismo dentro de una CPU generalista han empujado a buscar arquitecturas más especializadas (Hennessy y Patterson, 2019).

Este cambio se nota especialmente en aplicaciones de inteligencia artificial. Una red neuronal no suele estar limitada por una única operación compleja, sino por miles o millones de operaciones simples repetidas: multiplicaciones, acumulaciones, accesos a memoria, activaciones y movimientos de datos. En una CPU estas operaciones se ejecutan siguiendo una arquitectura muy flexible, pero no siempre eficiente. En cambio, una FPGA permite construir una ruta de datos a medida para una tarea concreta, explotando paralelismo espacial y reutilizando recursos de forma controlada (Wu et al., 2021).

Por este motivo, muchos sistemas empotrados actuales se plantean como sistemas heterogéneos: una parte del chip se dedica al control y a la ejecución de software convencional, mientras que otra parte implementa aceleradores específicos. En este trabajo esa división aparece de forma natural: el procesador ARM prepara los datos, controla las transferencias y muestra los resultados, mientras que la lógica programable ejecuta la inferencia del modelo.

2.1. Arquitectura Zynq-7000 y Comunicaciones AXI

La familia Zynq-7000 combina en un mismo circuito integrado un sistema de procesamiento (*Processing System*, PS) y lógica programable (*Programmable Logic*, PL). En el caso de la Zybo utilizada en este trabajo, el dispositivo es un Zynq-7010. La parte PS incluye un ARM Cortex-A9 de doble núcleo, memoria DDR y periféricos habituales de un sistema empotrado. La parte PL equivale a una FPGA de la familia 7-series, donde se pueden integrar bloques hardware propios.

Esta arquitectura es especialmente interesante porque evita tener una FPGA

aislada. El procesador y la lógica programable comparten el mismo sistema y se comunican mediante buses AMBA AXI (AMD, 2025b). AXI4-Lite se utiliza normalmente para acceder a registros de control, por ejemplo para arrancar un IP o leer su estado. AXI4-Stream, en cambio, está pensado para transportar datos como un flujo continuo, sin direcciones por cada palabra. Por último, los puertos de alto rendimiento del Zynq permiten que un DMA lea y escriba directamente en memoria DDR, descargando al procesador de copiar los datos uno a uno.

Esta separación entre control y datos es clave en aceleración. El ARM no debe enviar cada pixel de forma manual al acelerador, porque entonces el coste de comunicación consumiría buena parte del beneficio. La solución habitual es preparar buffers en DDR, limpiar o invalidar la caché cuando sea necesario, y dejar que el AXI DMA mueva bloques completos entre memoria y FPGA.

La limitación principal de la Zybo no está en el concepto de arquitectura, sino en el tamaño del dispositivo. Según la hoja de datos de la familia Zynq-7000, el Zynq-7010 dispone de 17.600 LUT, 35.200 flip-flops, 60 bloques BRAM de 36 Kb y 80 bloques DSP (Xilinx, 2018). Estas cifras son suficientes para aprender y prototipar, pero obligan a cuidar mucho el tamaño de una red convolucional.

2.2. Síntesis de Alto Nivel (HLS)

Tradicionalmente, una FPGA se programaba describiendo el hardware con VHDL o Verilog. Esto da mucho control, pero también obliga a pensar en registros, ciclos de reloj, máquinas de estados y rutas de datos desde el principio. La Síntesis de Alto Nivel, o HLS, intenta elevar ese nivel de abstracción: el diseñador describe el algoritmo en C o C++, y la herramienta genera una implementación RTL que después puede integrarse como IP en Vivado.

En el caso de AMD/Xilinx, Vitis HLS sintetiza funciones C/C++ a RTL para implementarlas en la lógica programable y permite exportarlas como IP de Vivado (AMD, 2023). Esto no significa que el código C se ejecute como software dentro de la FPGA. La herramienta analiza bucles, arrays, dependencias de datos y tipos, y a partir de ahí construye hardware: memorias, registros, multiplexores, operadores aritméticos y lógica de control.

Las directivas HLS son una parte importante del flujo. Con PIPELINE se puede intentar que un bucle acepte nuevos datos cada pocos ciclos, aunque las operaciones anteriores sigan avanzando por la ruta de datos. Con UNROLL se pueden duplicar operaciones para ejecutar varias iteraciones en paralelo. Con directivas de interfaz se decide si una función se conectará por AXI4-Lite, AXI4-Stream o memoria. Estas decisiones no son detalles menores: afectan directamente a la latencia, al consumo de LUT, al uso de DSP, a la BRAM y a la posibilidad de que Vivado coloque el diseño en la FPGA.

Por eso HLS no elimina el diseño hardware, sino que cambia dónde se toman muchas decisiones. En un proyecto pequeño puede acelerar mucho el desarrollo; en un proyecto ajustado en recursos, como una CNN en una Zynq-7010, sigue siendo imprescindible leer los informes de síntesis, entender los cuellos de botella y ajustar el código o las directivas.

2.3. hls4ml y generación de modelos para FPGA

hls4ml (FastML Team, 2025) es una herramienta de codiseño hardware-software orientada a traducir modelos de aprendizaje automático a implementaciones para FPGA o ASIC. En lugar de escribir a mano cada capa de una red neuronal en C++ para HLS, hls4ml parte de modelos entrenados en herramientas habituales de machine learning y genera un proyecto HLS con la topología, los pesos y las funciones necesarias para la inferencia (Fahim et al., 2021).

Su flujo encaja muy bien con este proyecto. Primero se entrena un modelo en TensorFlow/Keras y se guarda en formato `.h5`. Después hls4ml interpreta esa red, crea una representación interna del grafo y genera código C++ sintetizable. La documentación de hls4ml describe esta separación entre frontends, que leen modelos de frameworks como Keras u ONNX, y backends, que generan código para herramientas como Vivado HLS, Vitis HLS o Intel HLS (Fast Machine Learning Lab, 2026).

La aportación de hls4ml no es solo convertir capas. También permite configurar la precisión de los datos, el factor de reutilización de operadores, la estrategia de síntesis y el tipo de entrada y salida del modelo. En una FPGA pequeña, estas opciones son decisivas. Reducir el ancho de palabra puede ahorrar LUT y BRAM, pero también puede degradar la precisión. Aumentar el factor de reutilización reduce área, pero aumenta latencia. Elegir una arquitectura de flujo puede ahorrar memoria en algunas capas, pero introduce FIFOs y lógica adicional que también cuentan para Vivado.

En este trabajo se ha usado hls4ml precisamente como puente entre el entrenamiento del modelo y la generación del IP. La red no se ha tratado como una caja negra: ha sido necesario ajustar canales, precisiones, buffers, interfaz AXI-Stream y configuración de recursos hasta conseguir que el diseño entrase en la Zybo.

2.4. IA en FPGAs con recursos limitados

El despliegue de modelos de visión artificial en sistemas empotrados está condicionado por dos restricciones: cálculo y memoria. En una GPU o en una FPGA grande se puede ejecutar una red convolucional con muchas capas y canales, pero en una Zynq-7010 cada capa adicional se nota en el uso de LUT, BRAM y lógica de control.

Por eso, cuando se trabaja con placas pequeñas, se suelen aplicar modelos compactos y técnicas de cuantización. La cuantización reduce la precisión de pesos y activaciones para que la inferencia pueda ejecutarse con enteros o punto fijo en lugar de coma flotante. Esta idea está ampliamente extendida en inferencia embebida: trabajos como el de Jacob et al. muestran que la aritmética entera puede reducir memoria y mejorar la eficiencia manteniendo una precisión razonable si se diseña correctamente el flujo de entrenamiento e inferencia (Jacob et al., 2018).

En este proyecto se ha seguido esa misma filosofía, aunque adaptada al flujo HLS. El modelo se entrena en coma flotante, pero la inferencia final se ejecuta con tipos de punto fijo (AMD, 2026a). Esta decisión permite que el diseño sea viable en

recursos, pero introduce una consecuencia importante: el comportamiento numérico de la FPGA no tiene por qué coincidir exactamente con el modelo original de Keras. Los redondeos, truncados y anchos de acumulación forman parte del diseño.

CIFAR-10 añade además una dificultad razonable para una placa de este tamaño. El dataset contiene 60.000 imágenes RGB de 32x32 repartidas en 10 clases, con 50.000 imágenes de entrenamiento y 10.000 de test (Krizhevsky, 2009). Frente a MNIST, que trabaja con dígitos en escala de grises, CIFAR-10 exige extraer características de color, textura y forma en imágenes muy pequeñas. Esto lo convierte en una prueba más interesante, pero también más exigente para una CNN reducida.

2.5. Herramientas utilizadas en el flujo

El proyecto se apoya en varias herramientas que cubren etapas distintas del desarrollo:

- **TensorFlow/Keras:** entrenamiento del modelo y exportación a formato `.h5`.
- **hls4ml:** conversión del modelo entrenado a código C++ sintetizable y generación del proyecto HLS.
- **Vitis HLS:** síntesis del código C++ a RTL y empaquetado del acelerador como IP.
- **Vivado:** integración del IP con el Zynq Processing System, AXI DMA e interconexiones AXI, además de síntesis, implementación y generación del bitstream.
- **Vitis:** desarrollo de las aplicaciones bare-metal que se ejecutan sobre el ARM, inicializan los drivers, preparan buffers y lanzan la inferencia.

Esta cadena de herramientas permite recorrer todo el camino desde el entrenamiento hasta la placa, pero también introduce un reto práctico: cada etapa genera artefactos intermedios y puede conservar resultados antiguos. Por eso la reproducibilidad y la organización de carpetas son parte importante del trabajo, no solo una cuestión estética.

2.6. Panorama industrial y proyectos de referencia

La transición hacia la computación heterogénea no es solo una respuesta académica al estancamiento del rendimiento de las CPUs. También se ha convertido en una estrategia industrial para reducir latencia y consumo en cargas muy concretas.

2.6.1. Microsoft Project Catapult

Uno de los referentes más conocidos en el uso de FPGAs a gran escala es Project Catapult de Microsoft. Este proyecto surgió para acelerar tareas de búsqueda

y clasificación en centros de datos (Putnam et al., 2014). La idea principal era añadir FPGAs a los servidores para obtener aceleradores reconfigurables capaces de adaptarse a distintos algoritmos sin sustituir toda la infraestructura.

<https://www.microsoft.com/en-us/research/project/project-catapult/>

2.6.2. NVDLA

NVDLA, anunciado por NVIDIA, representa otro enfoque: una arquitectura abierta y configurable para inferencia de aprendizaje profundo (NVIDIA, 2026). Aunque está más cerca de un acelerador dedicado que de una FPGA general, es relevante porque refleja la misma tendencia: construir hardware específico para redes neuronales, con especial atención a tipos de datos reducidos y eficiencia.

<https://nvdla.org/>

2.6.3. BNN-PYNQ-ZYBO

A una escala más cercana a este trabajo, el repositorio *BNN-PYNQ-ZYBO* adapta el proyecto BNN-PYNQ (Umuroglu et al., 2017) para la Zybo-Z7-10. No es exactamente la Zybo Legacy utilizada aquí, pero sí trabaja con una placa basada en el mismo tamaño de dispositivo, el Zynq-7010. El propio repositorio indica que incluye topologías pensadas específicamente para la Zybo-Z7-10, con redes de pesos y activaciones de 1 bit, scripts de síntesis para esa placa y soporte para conjuntos como CIFAR-10 y MNIST (fcaspe, 2020).

Este tipo de proyecto es útil como referencia porque confirma una idea que también aparece durante este TFG: en un Zynq-7010, la clave no es intentar llevar una red grande sin cambios, sino diseñar una red muy compacta, cuantizada y ajustada al hardware disponible. La diferencia es que este trabajo no parte de una red binaria ya preparada, sino que recorre el flujo completo desde TensorFlow/Keras hasta hls4ml, Vitis HLS, Vivado y la aplicación bare-metal.

En conjunto, el estado de la cuestión muestra que el proyecto se sitúa en una línea muy actual: la necesidad de mover inferencia de IA hacia hardware especializado y cercano al dato. La Zybo no puede competir con aceleradores modernos, pero sí permite estudiar a pequeña escala los mismos problemas: movimiento de datos, cuantización, generación de IPs, integración AXI y equilibrio entre precisión, latencia y recursos.

Descripción del Trabajo

El desarrollo de este sistema heterogéneo se plantea de forma modular e incremental para aislar posibles fallos. A continuación, se detalla la metodología de diseño, dividida en cinco fases técnicas, partiendo desde la configuración inicial hasta la inferencia del modelo neuronal y su evaluación.

3.1. Configuración del entorno y preparación (Fase 1)

Antes de comenzar con la lógica del proyecto, es imperativo establecer un entorno de trabajo sólido. Las herramientas seleccionadas son AMD Vivado (para el diseño hardware) y AMD Vitis (para el desarrollo del software en C/C++). Esta primera toma de contacto tendrá como fin crear una aplicación funcional básica para comprobar el correcto funcionamiento del entorno de trabajo, comunicación con la placa de desarrollo y comprobación de ejecución. La lógica programable se configurará para encender unos LEDs con unos pulsadores. Con esto comprobaremos de forma fácil que la parte FPGA de la placa se llega a programar y que es independiente de la CPU. Después el sistema de procesamiento (PS) tendrá que mostrar unos mensajes por la terminal serie, leer los interruptores físicos de la placa dos veces, interpretar los números binarios de los interruptores, sumarlos y mostrar el resultado. Esto hace que comprobemos que nuestro programa en lenguaje C pueda leer correctamente los GPIO de la placa y que hay comunicación por el puerto serie para la terminal.

Los pasos iniciales son:

1. **Instalación de Board Files y constraints:** Descargar los archivos de definición de la placa Zybo Zynq-7010 desde el repositorio oficial de Digilent (Digilent, 2025) e instalarlos en Vivado. Esto preconfigura los relojes, las memorias DDR y los pines del SoC específicos de esta placa. Además necesitamos un fichero de constraints para que Vivado sepa que pines físicos vamos a utilizar y de qué manera. Este paso nos sirve para el resto de las implementaciones que haremos.
2. **Diseño del Sistema Base:** En Vivado, se crea un Block Design donde se

instancia el IP "ZYNQ7 Processing System". Se ejecuta la automatización de bloques (Block Automation) para aplicar la configuración de la Zybo. Se añade un bloque `.AXI GPIO` conectado a los LEDs de la placa.

3. **Generación y Exportación:** Se genera el Bitstream (archivo binario que configura la FPGA) y se exporta la plataforma hardware (`.xsa`).
4. **Validación Software:** En Vitis, se crea un proyecto de aplicación baremetal (sin sistema operativo). Se escribe un código sencillo en C que imprime texto por el puerto serie (UART) y hace parpadear los LEDs mediante registros AXI-Lite, demostrando que el procesador y la FPGA están vivos y comunicándose.

3.2. Comunicación PS-PL (Fase 2)

Para el proyecto de aceleración, la CPU no puede enviar datos a la FPGA dato a dato. Se necesita un canal directo a la memoria RAM.

1. **Habilitación de Puertos HP:** En Vivado, se modifica la configuración del bloque Zynq para activar un puerto esclavo "High Performance" (`S_AXI_HP0`).
2. **Integración del IP AXI DMA:** Se añade el bloque `.AXI Direct Memory Access`. Se configura deshabilitando la opción Scatter-Gather para mantener el diseño simple inicialmente (AMD, 2025a).
3. **Loopback de Prueba:** Para validar el DMA antes de introducir algoritmos complejos, se conecta el puerto de salida del DMA (`M_AXIS_MM2S`) directamente a su puerto de entrada (`S_AXIS_S2MM`) mediante un bloque FIFO.
4. **Software del DMA:** En Vitis, se configuran las cachés (usando `Xil_DCacheFlushRange` y `Xil_DCacheInvalidateRange` para garantizar la coherencia de los datos entre la caché del ARM y la memoria principal) y se lanzan transferencias de arrays de prueba, validando que el hardware es capaz de mover bloques de datos de forma autónoma.

3.3. Desarrollo del acelerador base en HLS: multiplicación de matrices (Fase 3)

En esta fase del proyecto se ha llevado a cabo el diseño, integración y evaluación de un acelerador hardware dedicado a la multiplicación de matrices, utilizando metodologías de codiseño HW/SW. El objetivo principal ha sido utilizar la FPGA con una tarea altamente paralelizable y cuantificar la mejora de rendimiento obtenida enfrentando los resultados de acelerar el cálculo con la FPGA contra realizar el cálculo de forma secuencial en SW.

3.3.1. Diseño del acelerador mediante Síntesis de Alto Nivel (HLS)

El desarrollo del núcleo IP se ha realizado utilizando la herramienta Vitis HLS, permitiendo describir el comportamiento del hardware a partir de un algoritmo en C++. Para maximizar el rendimiento del silicio generado, se han aplicado directivas de optimización (Pragmas):

- **Pipeline (`#pragma HLS PIPELINE`):** Implementado en los bucles internos para permitir el solapamiento de instrucciones, reduciendo el intervalo de iniciación a un ciclo de reloj y maximizando el throughput de la ruta de datos.
- **Unroll (`#pragma HLS UNROLL`):** Aplicado para desenrollar total o parcialmente las operaciones de multiplicación y acumulación (MAC), forzando a la herramienta de síntesis a instanciar múltiples bloques DSP (Digital Signal Processors) operando en paralelo.

Las interfaces del IP core se han configurado bajo el estándar AMBA AXI4: puertos AXI4-Stream para el flujo de datos de alta velocidad e interfaces AXI4-Lite para la configuración y control desde el procesador principal.

3.3.2. Integración a Nivel de Sistema y Arquitectura de Memoria

El bloque generado se ha integrado en el Block Design de Vivado formando una ruta de procesamiento completa. Para el movimiento de datos entre la memoria principal y el acelerador hardware se ha instanciado un controlador DMA en modo simple.

Para evitar cuellos de botella y bloqueos en el bus del sistema, se ha diseñado una arquitectura de memoria con separación estricta de dominios:

- El canal de lectura (MM2S) del DMA se ha enrutado de forma exclusiva a través del puerto de alto rendimiento S_AXI_HP0 del Zynq.
- El canal de escritura (S2MM) del DMA se ha canalizado independientemente a través del puerto S_AXI_HP1.

Esta topología garantiza canales de comunicación dedicados e independientes para la transferencia bidireccional continua de matrices.

3.3.3. Implementación software y coherencia de caché

El control del sistema se ha desarrollado mediante una aplicación bare-metal en lenguaje C/C++ utilizando Vitis IDE. El código ejecuta la inicialización de los controladores de hardware y gestiona las secuencias de disparo del DMA y del núcleo HLS.

Un factor crítico abordado en la capa de software ha sido la coherencia de caché. Dado que el controlador DMA accede directamente a la memoria física y no tiene visibilidad sobre las cachés L1/L2 del procesador ARM, ha sido imperativo ejecutar rutinas de limpieza (Xil_DCacheFlushRange) antes de las transmisiones e invalidaciones (Xil_DCacheInvalidateRange) previas a las lecturas, asegurando así la integridad de los operandos y resultados (AMD, 2026b).

3.3.4. Evaluación de rendimiento (Benchmarking)

Para cuantificar la ganancia de velocidad (speedup) real del sistema, se ha leído de forma directa los registros de hardware del Global Timer de 64 bits integrado en el procesador Cortex-A9. Esta técnica evita la sobrecarga de las librerías de tiempo estándar de C y proporciona una resolución del orden de nanosegundos (basada en el reloj de 333.33 MHz del temporizador).

Los resultados de las pruebas de rendimiento demuestran un comportamiento dual dependiente de la carga de datos:

- **Para matrices de dimensiones reducidas ($N = 4$):** La ejecución puramente en software (CPU) presenta una latencia menor. Esto se debe al overhead o penalización temporal implícita en la configuración de los registros AXI-Lite del DMA, el vaciado de las cachés y la latencia propia del bus de interconexión física. Para un volumen de datos tan pequeño, el tiempo de comunicación supera al tiempo de cómputo.
- **Para matrices escaladas ($N = 32$):** A medida que la complejidad computacional del algoritmo crece de forma exponencial, el modelo de ejecución secuencial del procesador ARM sufre una degradación de rendimiento. Por el contrario, el núcleo acelerador en la FPGA absorbe esta carga gracias al paralelismo masivo de los DSPs instanciados. En este escenario, el tiempo de procesamiento hardware diluye completamente el overhead de la comunicación por bus, arrojando un speedup significativo y validando empíricamente la utilidad de la aceleración por hardware en operaciones algebraicas intensivas.

	Aceleración FPGA (μs)	Puro SW (μs)
N=4	5.3680	3.3811
N=32	44.8988	1338.6451

Tabla 3.1: Resultados en tiempo de la multiplicación de matrices paralelas

3.4. Diseño e implementación del modelo de IA (Fase 4)

Aunque el planteamiento inicial contemplaba un modelo Perceptrón Multicapa (MLP) para el conjunto de datos MNIST (28x28 píxeles en escala de grises), la

disponibilidad de herramientas avanzadas de Síntesis de Alto Nivel (HLS) permitió elevar el alcance del proyecto. El diseño pivotó hacia una Red Neuronal Convolutiva (CNN) capaz de clasificar imágenes a color de 32x32x3 píxeles correspondientes al dataset CIFAR-10.

Dado que la plataforma hardware objetivo es la placa Zybo la cual posee recursos lógicos muy limitados, el flujo de diseño requirió técnicas de optimización extremas.

3.4.1. Entrenamiento y selección del modelo

El entrenamiento del modelo se realizó en Python con TensorFlow/Keras utilizando un pequeño código en Python para obtener el modelo entrenado en un formato *.h5. (TensorFlow, 2022). La red elegida mantiene una estructura sencilla, pero suficiente para extraer características espaciales de las imágenes CIFAR-10:

- Dos bloques iniciales formados por convolución 2D y *MaxPooling*, para reducir progresivamente el tamaño espacial de la imagen (Keras, 2026).
- Una tercera convolución 2D para extraer características de mayor nivel.
- Una convolución 1×1 para mezclar canales sin aumentar demasiado el coste de cálculo (Lin et al., 2014).
- Una última convolución 1×1 con 10 salidas, una por cada clase de CIFAR-10.
- Una capa final *GlobalAveragePooling2D*, que evita utilizar capas densas grandes y reduce de forma importante el número de parámetros.

Esta decisión fue importante porque las capas densas, aunque son sencillas de programar, tienden a consumir demasiada memoria y demasiada lógica cuando se llevan directamente a hardware. En cambio, una arquitectura totalmente convolutiva permite conservar la idea de clasificación de imágenes y, al mismo tiempo, mantener el tamaño del circuito dentro de unos límites razonables.

Durante el desarrollo se probaron varias combinaciones de canales y precisiones. El primer enfoque que cabía en la placa era demasiado pequeño y ofrecía resultados de clasificación poco útiles. Después se intentó aumentar el tamaño del modelo y forzar un mayor uso de DSPs, pero en la práctica el diseño empezó a fallar en implementación por exceso de LUTs o por problemas de empaquetado de *slices*. Finalmente se encontró una variante intermedia con la siguiente distribución de canales:

8 / 14 / 32 / 24 / 10

Esta versión consiguió un equilibrio más razonable: el modelo seguía siendo suficientemente pequeño para entrar en la Zynq-7010, pero mantenía una precisión aceptable para el alcance del proyecto.

3.4.2. Conversión del modelo a HLS

Una vez obtenido el modelo en formato *.h5, se utilizó hls4ml para traducir la red neuronal a código C++ compatible con Vitis HLS. Este paso no consiste solo en convertir la topología de Keras, sino también en decidir cómo se representarán los datos dentro del hardware.

El modelo original trabaja en coma flotante, pero esto no es adecuado para una FPGA pequeña como la Zybo. Por este motivo se empleó aritmética de punto fijo mediante tipos *ap_fixed* (AMD, 2026a). La entrada y los pesos se redujeron a formatos de pocos bits, mientras que las acumulaciones internas se dejaron con algo más de margen para evitar pérdidas graves de precisión. A nivel de configuración, se utilizó una estrategia orientada a recursos:

- **Strategy = Resource:** Prioriza reutilizar operadores frente a generar más paralelismo del necesario.
- **ReuseFactor elevado:** Permite que las mismas unidades de cálculo se reutilicen varias veces, reduciendo el área a cambio de aumentar la latencia.
- **ConvImplementation = LineBuffer:** Hace que las convoluciones trabajen con buffers de línea, una técnica más adecuada para procesar imágenes en flujo.
- **FIFO_opt activado:** Reduce el tamaño de algunas colas internas generadas por hls4ml.

También se probó a forzar que más multiplicaciones se implementasen usando bloques DSP. Aunque en teoría esto podía liberar LUTs, en este caso no resultó ser la solución principal. Al tratarse de multiplicaciones pequeñas, con bastante reutilización y mucha lógica de control alrededor, mover operaciones a DSP no ha reducido el uso total de LUTs. Por ello, la versión final se generó sin forzar explícitamente el uso de DSPs para todas las multiplicaciones.

3.4.3. Adaptación del IP core para AXI DMA

El código generado por hls4ml no se conectó directamente al sistema, sino que se encapsuló en un *wrapper* específico. Este envoltorio adapta la interfaz del modelo neuronal a una comunicación AXI4-Stream, que es la interfaz natural para conectarlo con el AXI DMA.

El IP core final expone tres tipos de señales (AMD, 2025b):

- **s_axis:** entrada AXI4-Stream por la que llegan los píxeles de la imagen desde memoria DDR. Cada palabra tiene 32 bits y representa un valor en formato Q16.16.
- **m_axis:** salida AXI4-Stream por la que se devuelven los 10 logits de clasificación, también codificados en Q16.16.

- **s_axi_CTRL:** interfaz AXI4-Lite de control, generada por Vitis HLS, que permite al procesador arrancar el IP mediante *ap_start* y comprobar señales como *ap_done*, *ap_idle* y *ap_ready*.

El *wrapper* realiza tres tareas. Primero lee 3072 palabras de entrada, correspondientes a una imagen de $32 \times 32 \times 3$. Después alimenta el núcleo generado por hls4ml. Por último, convierte los 10 resultados del modelo a palabras AXI Stream y marca con la señal *TLAST* el último dato de salida. Esto es necesario para que el canal S2MM del DMA sepa dónde termina la transferencia.

3.4.4. Integración en Vivado

En Vivado se creó el diseño final con una arquitectura que mantiene la misma idea que se había validado en las fases anteriores, el procesador ARM controla el sistema y la FPGA realiza el cálculo intensivo.

El *Block Design* integra los siguientes bloques principales:

- **ZYNQ7 Processing System:** Contiene el procesador ARM Cortex-A9, la memoria DDR y los puertos AXI necesarios para controlar el sistema.
- **AXI DMA:** Mueve datos entre la memoria DDR y el acelerador, evitando que la CPU tenga que escribir cada dato de forma individual.
- **cifar10_axis_wrapper:** IP core generado con Vitis HLS que ejecuta la inferencia del modelo CIFAR-10.

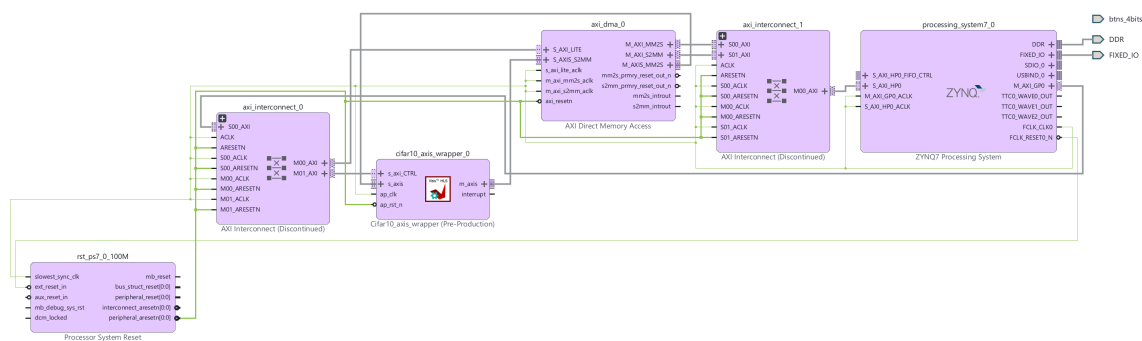


Figura 3.1: Diseño de bloques para la aceleración CNN de CIFAR-10

La conexión de datos se realiza de forma directa: el canal MM2S del DMA envía la imagen al puerto *s_axis* del IP, y el puerto *m_axis* del IP devuelve los resultados al canal S2MM del DMA. La configuración y el control del DMA y del IP se realizan desde el procesador mediante AXI4-Lite.

Uno de los puntos más delicados de esta fase fue el ajuste de recursos. La Zybo Zynq-7010 dispone de 17600 LUTs, por lo que pequeños cambios en la arquitectura del modelo podían hacer que el bitstream dejase de generarse. Se probaron varias variantes que fallaban por exceso de LUTs o por imposibilidad de empaquetar correctamente las instancias en los slices.

3.4.5. Aplicación bare-metal en Vitis

La aplicación de Vitis se encarga de preparar los datos, arrancar la inferencia y mostrar los resultados por la terminal serie. Para simplificar las pruebas se incluyeron imágenes CIFAR-10 ya convertidas a formato Q16.16 dentro de un fichero de cabecera. Así, cambiando una macro se puede seleccionar la imagen que se envía a la FPGA para probar con todas las clases de imágenes.

También se puede seleccionar una imagen previamente convertida a un array de Q16.16 que se puede incluir en nuestro código para ejecutar la inferencia sobre esta. Para ello, se ha incluido un script de Python para transformar imágenes en este formato específico, como se detalla en el capítulo 4 para facilitar el uso de esta aplicación de inferencia.

La secuencia de ejecución para comenzar la inferencia acelerada por FPGA es la siguiente:

1. Inicializar el controlador AXI DMA y comprobar que trabaja en modo simple.
2. Inicializar el driver del IP *cifar10_axis_wrapper*.
3. Copiar la imagen seleccionada al buffer de entrada y limpiar el buffer de salida.
4. Hacer flush de la caché para que el DMA vea los datos correctos en memoria DDR.
5. Iniciar primero la transferencia S2MM, para que el DMA esté preparado para recibir los resultados.
6. Arrancar el IP HLS mediante la señal *ap_start*.
7. Iniciar la transferencia MM2S para enviar la imagen hacia la FPGA.
8. Esperar a que terminen MM2S, S2MM y el propio IP.
9. Invalidar la caché del buffer de salida antes de leer los logits.

La gestión de caché es crítica, si el procesador conserva datos antiguos en caché, el DMA puede leer una imagen incorrecta o el software puede imprimir resultados que no corresponden con la inferencia recién ejecutada. Por eso se utilizan llamadas explícitas a *Xil_DCacheFlushRange* e *Xil_DCacheInvalidateRange*.

3.4.6. Interpretación de la salida

El modelo no devuelve directamente el nombre de una clase, sino 10 valores numéricos llamados logits. Cada logit representa la puntuación asociada a una clase de CIFAR-10. La predicción se obtiene seleccionando el índice con el valor más alto, es decir, aplicando una operación *argmax*.

Para que la salida por terminal sea más comprensible, la aplicación calcula una confianza aproximada mediante una versión simplificada de *softmax* (Guo et al., 2017). Primero se resta el mayor logit a todos los valores para evitar desbordamientos. Después se aproxima la exponencial y se normalizan los resultados para que

sumen el 100 %. Esta confianza no forma parte del IP hardware, sino que se calcula en el programa de test para facilitar la lectura de los resultados.

Los valores se imprimen en formato hexadecimal, entero con signo y aproximación decimal. Por ejemplo, un valor Q16.16 divide la palabra de 32 bits en 16 bits para la parte entera y 16 bits para la parte fraccionaria. Por tanto, para interpretar un logit se toma el entero con signo y se divide entre 65536.

3.4.7. Problemas encontrados durante la integración

Durante la integración apareció un problema especialmente importante: Vivado y Vitis podían conservar versiones antiguas del IP core aunque se hubiese generado una versión nueva con el mismo nombre. Esto provocaba que el bitstream pareciera correcto, pero la placa seguía ejecutando una versión anterior del acelerador y los resultados de inferencia eran incoherentes.

La solución pasa por limpiar los productos generados en Vivado, retirar el IP antiguo del diseño, volver a añadir el repositorio del IP actualizado, generar de nuevo el bitstream, exportar el fichero XSA. En Vitis la solución es eliminar la plataforma y aplicación, volverlas a generar de cero añadiendo el correspondiente fichero XSA y lanzar build. Tras esta limpieza, la ejecución sobre la placa empezó a devolver logits coherentes con el modelo entrenado.

Otro problema muy común es el límite de caracteres de Windows para las rutas de ficheros. Cuando una ruta supera los 250 caracteres, la build de Vitis o Vivado falla, teniendo que mover algunos ficheros a rutas más cortas.

Como algunos ficheros autogenerados también superan este límite, siendo imposible de mover estos ficheros a rutas más cortas, la solución ha sido utilizar el comando *subst* para asociar la ruta del proyecto de Vivado/Vitis a una letra de unidad de Windows, haciendo la ruta lo más corta posible.

3.5. Pruebas y benchmarking (Fase 5)

La última fase del proyecto se planteó como una fase de validación. Después de tener el IP CIFAR-10 integrado en Vivado y una aplicación bare-metal capaz de comunicarse con él, era necesario preparar un entorno de pruebas que permitiese medir tiempos, repetir ejecuciones y comprobar que los datos enviados a la FPGA eran exactamente los esperados.

3.5.1. Preparación de pruebas de recursos

Además de la primera prueba de rendimiento correspondiente a la fase 3, donde se implementó una multiplicación de matrices acelerada mediante hardware FPGA generada por Vitis HLS, es de especial interés medir el uso de área hardware que hace esta herramienta y compararlo con el que hace el tradicional diseño en VHDL. Ambas fueron configuradas para explotar paralelismo mediante varios multiplicadores simultáneos y para almacenar datos en memoria interna de la FPGA. Esta comparación no buscaba mejorar la funcionalidad de la multiplicación, sino obser-

var cómo repartía recursos Vivado cuando el mismo problema se describía desde dos niveles de abstracción distintos.

3.5.2. Generación de imágenes de prueba en formato Q16.16

Para probar la red CIFAR-10 sobre la placa no bastaba con tener imágenes en formato convencional. El IP espera recibir 3072 palabras de 32 bits, una por cada canal RGB de una imagen de 32x32 píxeles. Por eso se prepararon scripts auxiliares que convierten una imagen en un fichero `.h` que puede incluirse directamente en la aplicación de Vitis.

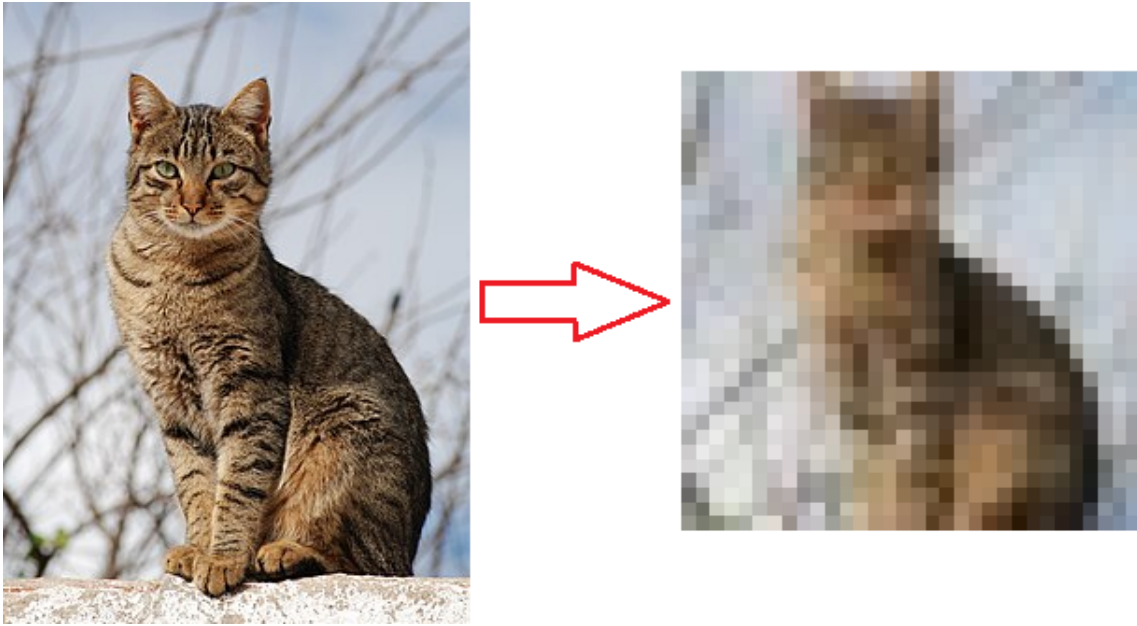


Figura 3.2: Imagen reducida con la herramienta `image_file_to_h.py`

El script principal es `image_file_to_h.py`. Este programa recibe una imagen indicada por el usuario, la lee mediante TensorFlow, la redimensiona a 32x32 y genera un array en formato Q16.16. El orden de los datos se mantiene como una secuencia RGB:

```
pixel0_R, pixel0_G, pixel0_B, pixel1_R, pixel1_G, pixel1_B, ...
```

Cada canal se normaliza desde el rango 0–255 hasta el rango $[0, 1]$ y después se escala a Q16.16 mediante la expresión:

$$\text{valor}_{\text{Q16,16}} = \text{round} \left(\frac{\text{canal}}{255} \cdot 65536 \right) \quad (3.1)$$

El script permite elegir entre varios modos de redimensionado, como `crop`, `stretch` o `pad`. También permite indicar una etiqueta esperada, cambiar el nombre que se muestra por terminal y guardar una copia de la imagen ya reducida. Esta última imagen fue útil durante las pruebas, porque permitía comprobar visualmente qué entrada estaba recibiendo realmente la red neuronal tras el escalado a 32x32.

3.5.3. Aplicaciones de inferencia y medición

Para la inferencia acelerada se encapsuló la comunicación con la FPGA en `fpga_cifar10_accel.c` y `fpga_cifar10_accel.h`. Estos ficheros inicializan el AXI DMA, inicializan el driver generado para el IP `cifar10_axis_wrapper`, mantienen la coherencia de caché y lanzan la transferencia de entrada y salida. La aplicación envía las 3072 palabras Q16.16 por el canal MM2S del DMA y recibe 10 palabras de salida, una por clase, por el canal S2MM.

La medida temporal se hizo con el Global Timer del Zynq-7010. Este temporizador permite medir tanto la ruta acelerada como la ruta software con el mismo criterio. En la ejecución FPGA se mide el tiempo total de una inferencia desde el punto de vista de la aplicación: limpieza de salida, coherencia de caché, DMA, ejecución del IP y lectura final del buffer. En la ejecución sobre el ARM se mide el tiempo de ejecución del core C++ generado por hls4ml sobre el Cortex-A9.

Para lograr este objetivo, se ha creado un nuevo espacio para el proyecto de Vitis, `5_cifar10_benchmark`, utilizando el fichero XSA y la implementación sobre FPGA de la fase 4. Esta separación deja la aplicación de la fase 4 centrada únicamente en la inferencia acelerada y reserva la fase 5 para la comparativa entre FPGA y ARM.

3.5.4. Criterios de interpretación

Las pruebas se interpretan siempre a partir de los logits devueltos por el modelo. La clase final se obtiene aplicando *argmax*, es decir, seleccionando el índice con mayor puntuación. El porcentaje de confianza que aparece por terminal se calcula después mediante una softmax aproximada en la aplicación, por lo que sirve como ayuda de lectura, no como salida directa del IP hardware.

En el capítulo 4 se separan dos tipos de resultados: por un lado, la validación funcional de la inferencia en la FPGA; por otro, el benchmark temporal frente a la ejecución del core hls4ml en el ARM. Esta separación evita mezclar el proceso de preparación del entorno con la discusión de los resultados finales.

Capítulo 4

Resultados y evaluación

En este capítulo se recogen los resultados obtenidos después de integrar las distintas partes del proyecto. No se pretende repetir aquí todo el proceso de desarrollo, descrito en el capítulo 3, sino analizar qué se ha conseguido finalmente y hasta qué punto los resultados cumplen los objetivos planteados al inicio del trabajo.

En primer lugar, se comparan los recursos hardware utilizados por una misma operación implementada mediante HLS y mediante VHDL. Después se analiza el encaje del acelerador CIFAR-10 dentro de la Zybo Z-7010. Por último, se estudia el tiempo de inferencia obtenido en FPGA frente a la ejecución del core C++ generado por hls4ml en el ARM.

4.1. Comparación de recursos entre HLS y VHDL

Antes de abordar el modelo CIFAR-10 completo, se utilizó la multiplicación de matrices de la fase 3 como prueba controlada. Esta prueba era más sencilla que la red neuronal, pero ya permitía estudiar una cuestión importante: si Vitis HLS generaba hardware razonable frente a una descripción manual en VHDL.

Los recursos obtenidos para ambas implementaciones se muestran en la siguiente tabla:

Diseño	LUT	FF	BRAM	DSP
VHDL	8263	4363	35	72
HLS	7723	6894	32	72

Tabla 4.1: Comparación de recursos entre la multiplicación de matrices en VHDL y en HLS

Ambas versiones utilizan 72 DSP, esto era esperable, ya que las dos implementaciones explotan el mismo tipo de paralelismo en la parte aritmética, pero nos asegura que ambas soluciones están alineadas. La diferencia está en cómo se organiza el resto del circuito. La versión generada con HLS usa 540 LUT menos y 3 BRAM menos que la versión VHDL, pero a cambio utiliza 2531 FF más.

Este resultado fue útil por dos motivos. Primero, porque confirmó que HLS no

estaba generando una implementación desproporcionada en área. Segundo, porque mostró una de las ventajas prácticas de la herramienta: el reparto entre memoria, registros y lógica de control se consigue con menos esfuerzo manual.

Durante el desarrollo de la versión VHDL apareció una primera implementación en la que no se inferían correctamente las BRAM y el uso de LUT se disparaba. En HLS este tipo de decisiones quedan más guiadas por la herramienta, lo cual no elimina la necesidad de revisar informes, pero sí reduce mucho el coste de llegar a una primera versión funcional.

4.2. Utilización de recursos del acelerador CIFAR-10

El resultado más delicado del proyecto fue conseguir que el modelo CIFAR-10 entrase en la Zybo Legacy. La Zynq-7010 integra una FPGA pequeña para implementar una red convolucional, por lo que el diseño estuvo limitado principalmente por el número de LUT disponibles. A lo largo del desarrollo se probaron varias configuraciones de modelo y de cuantización hasta encontrar una versión que Vivado pudiera colocar y rutear correctamente.

La versión final utilizada en placa fue el IP generado a partir del modelo:

`v4_8_14_32_24_final`

Tras integrarlo en Vivado, el diseño completo obtuvo la utilización mostrada en la siguiente tabla:

Recurso	Uso en el diseño final
LUT	16117
FF	17917
BRAM	37,5
DSP	6

Tabla 4.2: Utilización de recursos del diseño CIFAR-10 final en la Zybo

El diseño queda cerca del límite de LUT, pero entra en la placa. Esto es importante porque varias versiones anteriores fallaban en Vivado con errores de sobreutilización o de empaquetado de slices. En cambio, los FF no llegaron a ser el recurso crítico y el uso de BRAM se mantuvo dentro de lo admisible para la placa.

Un aspecto llamativo es el bajo uso de DSP (AMD, 2023). Durante el proyecto se intentó aumentar su utilización para descargar lógica combinacional, pero no siempre es posible trasladar automáticamente el coste de LUT a DSP. La decisión depende de los tipos de datos, de los anchos de palabra, de las directivas HLS, del factor de reutilización y de cómo Vivado termina mapeando las operaciones. En este caso, el límite real lo marcó la lógica auxiliar y de control asociada a la red, no solo las multiplicaciones.

4.3. Validación funcional de la inferencia

Una vez generado el bitstream, se realizaron pruebas con varias imágenes de prueba.

La primera ejecución se hizo sobre imágenes de test que entrega el propio dataset de CIFAR-10, las cuales se han tenido que transformar al formato Q16.16 normalizado.

La ejecución sobre todas estas imágenes ha sido la siguiente:

Img. enviada	Pred.	Plane	Auto	Bird	Cat	Deer	Dog	Frog	Horse	Ship	Truck
Plane	Plane	71.17 %	1.35 %	11.78 %	0.60 %	0.75 %	0.22 %	0.28 %	0.03 %	13.61 %	0.21 %
Auto	Auto	3.59 %	60.70 %	0.26 %	2.88 %	0.01 %	10.63 %	0.32 %	6.72 %	0.02 %	14.88 %
Bird	Truck	5.21 %	2.07 %	21.93 %	6.79 %	19.49 %	9.05 %	2.06 %	1.80 %	1.35 %	30.24 %
Cat	Cat	0.53 %	0.32 %	1.19 %	53.14 %	2.07 %	15.78 %	26.00 %	0.22 %	0.45 %	0.30 %
Deer	Deer	32.07 %	0.54 %	15.93 %	2.48 %	34.68 %	2.32 %	0.93 %	1.53 %	8.30 %	1.22 %
Dog	Frog	0.43 %	1.65 %	4.79 %	9.69 %	17.14 %	18.70 %	32.13 %	14.99 %	0.19 %	0.28 %
Frog	Frog	0.06 %	0.13 %	6.18 %	5.56 %	3.72 %	1.17 %	83.11 %	0.02 %	0.04 %	0.01 %
Horse	Truck	0.04 %	20.13 %	0.04 %	0.30 %	0.48 %	5.63 %	0.80 %	35.43 %	0.02 %	37.13 %
Ship	Auto	14.65 %	71.26 %	0.25 %	0.02 %	0.03 %	0.01 %	0.09 %	0.01 %	12.09 %	1.60 %
Truck	Truck	0.17 %	9.78 %	0.06 %	0.11 %	0.02 %	0.35 %	0.12 %	0.07 %	0.24 %	89.08 %

Tabla 4.3: Resultados de inferencia para 10 imágenes CIFAR-10 mostrando la confianza por clase

Con estos resultados, podemos decir que la inferencia en la Zybo funciona de forma real, aunque el modelo no es robusto. Teniendo en cuenta que el modelo ha sido optimizado para que entre de manera ajustada en los recursos que tiene la Z-7010, los resultados son aceptables para el estudio de este proyecto y poder verificar que en efecto, podemos ejecutar inferencia.

Muchos de los fallos de predicción que vemos, están casi empatados con la clase real:

- La inferencia sobre **Horse** está casi empatada con **Horse**(35.43 %) y **Truck**(37.13 %).
- La inferencia sobre **Dog** se confunde con **Frog**(32.13 %), pero reparte bastante confianza entre **Deer**(17.14 %), **Dog**(18.70 %) y **Horse**(14.99 %).
- La inferencia sobre **Deer** acierta aunque con un margen muy pequeño frente a **Plane**(32.07 %).

Estos repartos de predicción apuntan a un modelo pequeño, cuantizado y limitado, no a una salida rota del acelerador. Esta interpretación encaja con el efecto habitual de la cuantización: reducir precisión y memoria hace viable la inferencia en hardware pequeño, pero puede estrechar el margen entre clases cuando el modelo no tiene demasiada capacidad (Jacob et al., 2018).

Hay clases que el modelo reconoce con mucha seguridad, esto nos indica que hay patrones aprendidos que se conservan correctamente en la FPGA.

Aun así, esta prueba no es decisiva para validar la precisión global del modelo porque son solo 10 imágenes. CIFAR-10 incluye 60.000 imágenes de 32x32 píxeles repartidas en 10 clases, con 50.000 imágenes de entrenamiento y 10.000 imágenes

de test (Krizhevsky, 2009). Por tanto, una validación completa debería ejecutar el conjunto de test, o al menos una muestra mucho más amplia y balanceada por clase. Esa evaluación queda fuera del alcance de este trabajo, donde se ha priorizado cerrar el flujo completo de inferencia en placa.

Además de la prueba con imágenes sacadas del dataset, se ha podido probar la inferencia sobre diferentes imágenes descargadas de internet transformándolas mediante la herramienta `image_file_to_h.py`. Teniendo en cuenta que la imagen se debe redimensionar a 32x32 píxeles, si queremos obtener una predicción correcta con una confianza alta, es importante revisar la imagen reducida y comprobar que el objeto no se difumina con el entorno, es suficientemente grande en la imagen o no hay ningún otro objeto en la imagen.

4.4. Benchmark de inferencia

El objetivo principal de rendimiento era comprobar si la FPGA reducía la latencia de la inferencia frente a una ejecución software en el ARM Cortex-A9. Para ello se midieron varias ejecuciones sobre la misma imagen y con el mismo temporizador hardware de la plataforma. En esta comparación se usó el mismo modelo generado por hls4ml en dos caminos distintos: por un lado, el IP sintetizado e integrado en la FPGA. Por el otro, el core C++ generado por hls4ml ejecutado directamente sobre el ARM (Fast Machine Learning Lab, 2026).

La siguiente tabla muestra los tiempos obtenidos:

Ruta de inferencia	Tiempo medio	Mejor tiempo
FPGA	2012 μs	2008 μs
CPU (ARM)	171215 μs	171161 μs

Tabla 4.4: Benchmark de inferencia CIFAR-10 entre acelerador FPGA y core C++ de hls4ml en ARM

Con estos datos, la aceleración media observada es:

$$\text{speedup} = \frac{171215}{2012} \approx 85,09 \quad (4.1)$$

Es decir, la inferencia acelerada en FPGA fue aproximadamente 85 veces más rápida que la ejecución del core C++ de hls4ml en el ARM. Este resultado es coherente con la naturaleza de cada ruta. En la FPGA, el modelo se convierte en hardware dedicado y puede explotar paralelismo a nivel de operaciones y de flujo de datos. En cambio, en el ARM se ejecuta una descripción C++ pensada para ser sintetizada por HLS, no una librería de inferencia optimizada para CPU (Sze et al., 2017).

Es importante destacar que el tiempo de la ruta FPGA incluye el uso real del sistema: preparación del buffer de salida, coherencia de caché, transferencia DMA, ejecución del IP HLS y lectura de los resultados. Por tanto, no se está midiendo únicamente el núcleo matemático del acelerador, sino la inferencia completa desde

el punto de vista de la aplicación bare-metal. Aun incluyendo este coste de comunicación, el tiempo se mantiene alrededor de los 2 ms.

La ruta ARM, por su parte, queda penalizada por varios motivos. El core generado por hls4ml utiliza tipos de dato de precisión fija, estructuras tipo `hls::stream` y plantillas C++ orientadas a describir hardware. Todas estas construcciones tienen sentido cuando el código se va a transformar en un circuito, pero no son la forma más eficiente de ejecutar una red neuronal en un Cortex-A9. Por esta razón, el resultado no debe interpretarse como una comparación contra la mejor implementación posible en CPU, sino como una comparación entre el acelerador hardware y la ejecución software directa del mismo flujo hls4ml.

Además del tiempo de ejecución, también se observaron diferencias en los valores de salida. Para la imagen de prueba externa de un perro, ambas rutas predijeron correctamente la clase *dog*, pero con logits y porcentajes de confianza distintos:

Ruta	Predicción	Logit dog	Confianza aprox.
FPGA	dog	4.2188	91.22 %
CPU (ARM)	dog	2.4219	48.24 %

Tabla 4.5: Comparación de salida entre la inferencia en FPGA y la ejecución hls4ml en ARM

Esta diferencia no invalida el benchmark. La clase final se obtiene con el índice del mayor logit, y en este caso ambas rutas llegan a la misma predicción. Lo que cambia es la separación entre la clase ganadora y el resto de clases. En la FPGA, el logit de *dog* queda mucho más separado del resto, y por eso la softmax aproximada muestra una confianza superior al 90 %. En el ARM, la salida sigue clasificando como *dog*, pero la separación con otras clases es menor y la confianza baja hasta aproximadamente el 48 %.

Hay que recordar que la confianza mostrada por terminal no es una salida directa del modelo hardware. El IP devuelve logits en Q16.16, y la aplicación calcula después una softmax aproximada para facilitar la lectura. Por tanto, una diferencia moderada entre logits puede transformarse en una diferencia grande en el porcentaje final. Este efecto es habitual en clasificadores neuronales: la softmax amplifica la distancia relativa entre las puntuaciones (Guo et al., 2017).

Las diferencias entre FPGA y ARM también son razonables desde el punto de vista numérico. Aunque ambas rutas parten del mismo modelo entrenado, no ejecutan exactamente la misma implementación aritmética. La FPGA trabaja con el hardware generado por Vitis HLS, con los anchos de palabra, redondeos y comportamientos de desbordamiento definidos durante la síntesis. El ARM, en cambio, ejecuta una versión C++ del modelo mediante tipos software que emulan precisión fija. Pequeñas diferencias de redondeo, truncado, orden de acumulación o conversión entre tipos pueden propagarse a través de las capas convolucionales y modificar la distancia entre los logits finales.

Por tanto, la conclusión principal del benchmark es doble. En rendimiento, la FPGA ofrece una mejora muy clara frente a ejecutar el core hls4ml en el ARM. En precisión funcional, ambas rutas mantienen la misma clase predicha para la imagen evaluada, aunque no producen exactamente los mismos logits ni la misma

confianza. Esto confirma que la aceleración hardware es efectiva, pero también que en un despliegue con cuantización y HLS conviene analizar siempre los logits y no solo el porcentaje de confianza impreso por la aplicación.

4.5. Valoración global

Los resultados muestran que el sistema heterogéneo funciona de extremo a extremo: el ARM prepara la imagen, la envía a la FPGA mediante AXI DMA, el IP HLS ejecuta la inferencia y la aplicación recupera los logits para obtener la clase final. El diseño no se queda en una simulación aislada, sino que se ejecuta sobre la placa real.

La mejora temporal frente a ejecutar el core hls4ml directamente en el ARM es muy clara, aunque no debe separarse del contexto de la placa utilizada. La placa Zybo tiene recursos muy ajustados para una CNN, por lo que el modelo final tuvo que ser pequeño y cuidadosamente cuantizado. A pesar de ello, conseguir una inferencia en torno a 2 ms en la FPGA confirma que el uso de hardware dedicado es una vía efectiva para reducir latencia en sistemas empujados.

La evaluación también deja una enseñanza importante: en proyectos HLS no basta con que el modelo sea correcto a nivel de código C++. La versión realmente desplegada es el hardware generado, y por eso hay que analizar recursos, temporización, comunicación con memoria y comportamiento numérico una vez integrado en la placa. Esta parte es una pieza clave para poder confiar en los resultados del sistema completo.

Conclusiones y trabajo futuro

El objetivo principal de este trabajo era diseñar y evaluar un sistema heterogéneo sobre la placa Digilent Zybo, utilizando el procesador ARM para controlar la aplicación y la FPGA para acelerar una tarea de cálculo intensivo. Después del desarrollo realizado, se puede afirmar que este objetivo se ha cumplido. El sistema final permite enviar una imagen desde el ARM, transferirla a la lógica programable mediante AXI DMA, ejecutar un modelo convolucional generado con hls4ml y recuperar los logits de salida para obtener la clase predicha.

El camino hasta llegar a esa versión final no ha sido lineal. La principal dificultad no estuvo solo en entrenar un modelo CIFAR-10, sino en conseguir que dicho modelo cupiese realmente en una Zynq-7010. Muchas configuraciones que parecían razonables desde el punto de vista del número de parámetros no eran viables después de pasar por Vivado, especialmente por el uso de LUT y por los problemas de empaquetado de slices. Esta parte del trabajo dejó claro que, en una FPGA pequeña, no basta con hablar de tamaño del modelo: la forma en que las operaciones se sintetizan y se colocan en el dispositivo es igual de importante.

5.1. Cumplimiento de objetivos

La primera parte del proyecto permitió poner en marcha la plataforma y entender el flujo completo Vivado/Vitis. Se validó la ejecución bare-metal sobre el ARM, se configuró la comunicación entre el sistema de procesamiento y la lógica programable, y se trabajó con AXI-Lite y AXI DMA. Estos pasos fueron necesarios para que la parte de inteligencia artificial no quedase aislada como un IP sin integración real.

La multiplicación de matrices de la fase 3 sirvió como punto intermedio. Fue una prueba más controlada que la CNN, pero suficiente para comprobar que Vitis HLS podía generar hardware competitivo y que el flujo ARM-DMA-FPGA funcionaba correctamente. Además, la comparación con una implementación VHDL ayudó a valorar el uso de HLS con más criterio. La herramienta no elimina la necesidad de entender el hardware, pero sí reduce mucho el esfuerzo necesario para llegar a una arquitectura paralela funcional.

En la fase 4 se consiguió generar e integrar un modelo CIFAR-10 completo. La

precisión del modelo se tuvo que equilibrar con los recursos disponibles en la Zybo. La versión final no es una red grande, ni pretende competir con modelos modernos de visión artificial, pero sí cumple el propósito del trabajo: demostrar una inferencia de IA ejecutándose en hardware dedicado dentro de una FPGA de recursos modestos.

Por último, la fase de evaluación permitió medir el beneficio de la aceleración. La inferencia en FPGA obtuvo un tiempo medio cercano a 2 ms, frente a unos 171 ms de la ejecución del core C++ generado por hls4ml en el ARM. Esto supone una mejora aproximada de 85 veces. Más allá del número exacto, lo relevante es que la aceleración se ha medido sobre la placa real e incluyendo el coste de comunicación con el IP.

5.2. Lecciones aprendidas

Una de las conclusiones más claras del proyecto es que HLS facilita el desarrollo, pero no lo convierte en automático. Las directivas, los tipos de datos y el factor de reutilización tienen un impacto directo en los recursos finales. En varias ocasiones fue necesario volver al modelo, modificar su arquitectura o ajustar la cuantización porque el diseño no entraba en la FPGA. En ese sentido, el trabajo con hls4ml fue especialmente interesante: acerca mucho el mundo del aprendizaje automático al diseño hardware, pero sigue exigiendo leer con cuidado los informes de síntesis, utilización y temporización.

También ha sido importante distinguir entre los distintos niveles de validación. Al principio, cuando la ejecución en placa devolvía resultados incoherentes, era tentador pensar en un fallo del modelo. Sin embargo, parte del problema venía de cachés de Vivado/Vitis y de versiones antiguas del IP que seguían apareciendo en la integración.

Otro punto práctico ha sido la importancia de mantener los proyectos ordenados. En un flujo con Vivado, Vitis, Vitis HLS, modelos entrenados, IPs generados y scripts auxiliares, es fácil acabar con carpetas repetidas o resultados antiguos mezclados con versiones nuevas. La separación entre distintas fases y en cada una de ellas una carpeta con un proyecto Vivado/Vitis, hace que sea más fácil de mantener y de explicar.

5.3. Limitaciones

La principal limitación del sistema es la propia placa. La Zybo Legacy es una plataforma muy útil para aprender y prototipar, pero la Zynq-7010 tiene un margen reducido para implementar CNNs. El modelo final tuvo que ser pequeño y muy ajustado en recursos. Esto afecta directamente a la precisión que se puede esperar y limita la complejidad de las imágenes que puede clasificar con robustez.

Otra limitación es que el benchmark se ha centrado en latencia de inferencia sobre imágenes concretas. Para una evaluación más completa sería necesario ejecutar un conjunto amplio de imágenes de prueba y obtener métricas como precisión global, matriz de confusión o comportamiento por clase. En este trabajo se priorizó cerrar

el flujo completo hardware-software y medir la aceleración real en placa.

También hay que tener cuidado al interpretar la confianza mostrada por terminal. El IP devuelve logits, no probabilidades. La confianza se calcula en la aplicación con una softmax aproximada para facilitar la lectura humana de los resultados. Por tanto, la clase predicha y los logits son más importantes que el porcentaje exacto de confianza.

5.4. Trabajo futuro

Una línea clara de trabajo futuro sería mejorar el modelo manteniendo la restricción de recursos. Para ello podrían explorarse técnicas como poda de pesos, cuantización consciente del entrenamiento (*quantization aware training*), arquitecturas separables en profundidad o una búsqueda más sistemática de anchos de palabra por capa. El objetivo sería aumentar la precisión sin volver a superar el límite de LUT y BRAM de la placa (Han et al., 2016; Howard et al., 2017).

También sería interesante estudiar mejor el uso de DSP. En el diseño final el consumo de DSP es bajo, mientras que las LUT siguen siendo el recurso más comprometido. Un trabajo futuro podría centrarse en forzar de forma más controlada el mapeo de multiplicaciones a DSP, ajustar los anchos de dato para favorecer ese mapeo y comparar distintas estrategias HLS. No siempre será posible trasladar lógica a DSP, pero merece una exploración más profunda.

Otra mejora natural sería automatizar el flujo de generación. Actualmente el proceso implica entrenar el modelo, exportarlo a hls4ml, generar el IP, integrarlo en Vivado, exportar el XSA y reconstruir la plataforma de Vitis. Parte de este flujo podría unificarse mediante scripts que limpien versiones antiguas, regeneren el IP y dejen preparada una carpeta final reproducible. Esto reduciría errores de caché o de rutas, que han sido una fuente real de problemas durante el proyecto.

Desde el punto de vista de la evaluación, el siguiente paso sería probar muchas más imágenes y no solo casos individuales. Una aplicación que lea imágenes desde una tarjeta SD, o incluso desde una cámara, permitiría evaluar el sistema de forma más parecida a un caso de uso real. También sería interesante medir consumo energético, ya que una de las razones de usar FPGA en Edge-AI no es solo la latencia, sino la eficiencia (Sze et al., 2017).

Por último, se podría comparar el acelerador FPGA con una implementación ARM más optimizada, por ejemplo usando NEON o librerías pensadas para microcontroladores y procesadores embebidos (Lai et al., 2018). Esto permitiría situar mejor el resultado frente a una ruta software diseñada específicamente para CPU. Aun así, el trabajo realizado ya muestra la idea principal: cuando el cálculo se adapta al hardware y se integra correctamente con el sistema, la FPGA puede reducir de forma notable la latencia de inferencia incluso en una placa de recursos limitados.

Introduction

The computing landscape is undergoing a major shift. Driven by the rapid growth of Artificial Intelligence, real-time data processing is no longer merely desirable; it has become a fundamental requirement. While the traditional approach relied on offloading neural network computation to large cloud data centers, current trends are moving toward Edge AI: bringing intelligence closer to the data source, directly onto the local device. This approach significantly reduces latency, improves user privacy by avoiding the transmission of sensitive data over the network, and reduces the load on communication infrastructures. (Zhou et al., 2019).

This is where FPGAs become a key enabling technology. Unlike traditional CPUs and GPUs, which process data either sequentially or through fixed forms of parallelism, FPGAs allow custom hardware architectures to be designed from the ground up, optimizing data flow and exploiting instruction-level parallelism. This is particularly relevant for addressing current physical limitations such as the Power Wall, where further increasing the operating frequency of conventional processors is no longer energy-efficient. (Hennessy y Patterson, 2019)

For these reasons, this Bachelor’s Thesis focuses on an in-depth exploration of hardware-software co-design. To carry this out, the Digilent Zybo platform will be used (Digilent, 2024), a development board that integrates the flexibility of an ARM Cortex-A9 processor for control-oriented tasks with the computational capabilities of a Xilinx/AMD 7-series FPGA for intensive processing workloads, all within the same chip. From an academic perspective, this platform provides an ideal environment for emulating industrial embedded systems, giving the designer fine-grained control over every clock cycle and every data transfer through high-performance buses such as AXI4.

Motivation

The demand for computational performance, mainly driven by Artificial Intelligence and Deep Learning algorithms, is increasing at a pace that general-purpose processors can no longer address efficiently under strict power-consumption constraints. Moving inference from the cloud to local embedded devices is no longer optional; it is the natural direction of the field.

The main technical challenge of this work, and what makes it particularly de-

manding, is to make the most of an FPGA with limited resources, only around 80 DSP blocks, in order to run Machine Learning models. This will be supported by modern High-Level Synthesis (HLS) tools, which help reduce the complexity traditionally associated with pure hardware design and make the development process more accessible and efficient.

Objectives

The main objective of this work is to design, implement, and evaluate a heterogeneous embedded system based on the Zynq-7010 SoC, capable of accelerating the inference of a lightweight Artificial Intelligence model. As a direct consequence, this work also aims to quantify the performance improvement achieved by the hardware implementation when compared with the execution of the same inference flow on the ARM microprocessor. To achieve this goal, the following specific objectives are defined:

1. **Platform bring-up:** Configure the Vivado/Vitis development environment from scratch for the Digilent Zybo board, validating the independent operation of both the ARM Cortex-A9 microprocessor and the FPGA.
2. **Communication infrastructure design:** Implement high-bandwidth communication channels between the Processing System and the Programmable Logic, using the AMBA AXI protocol and Direct Memory Access (DMA) controllers.
3. **Acceleration of a baseline computation:** Develop a basic linear algebra IP core in C++, specifically a matrix multiplication accelerator, and synthesize it into hardware using HLS in order to validate the data flow between the processor and the FPGA.
4. **AI implementation under limited resources:** Adapt a neural network model, specifically a CIFAR-10 Convolutional Neural Network, to the limited resources of the Zynq-7010, generating its hardware implementation and developing the control software required to provide the input data.
5. **Performance evaluation (Benchmarking):** Compare execution times, in terms of latency, by running the neural network inference on the CPU alone and comparing it with the FPGA-assisted execution.

Work plan

To organize the development process and meet the project milestones, the work is divided into the following incremental phases:

Phase 1. Research and familiarization: Review of the state of the art in Zynq-based systems, AXI buses, and High-Level Synthesis. Installation of the required tools (Vivado and Vitis) and execution of a "Hello World" application on the board.

Phase 2. PS-PL communication: Configuration of control registers through AXI-Lite and setup of the memory subsystem using AXI DMA. Development of bare-metal C drivers to transfer data from RAM to the FPGA.

Phase 3. Development of the baseline accelerator in HLS: Implementation of matrix multiplication in C/C++. Use of HLS pragmas such as PIPELINE and UNROLL to optimize the generated hardware. Integration of the resulting IP block into the Vivado design together with the DMA.

Phase 4. Design and implementation of the AI model: Training and/or export of a Convolutional Neural Network model for CIFAR-10 image classification. Generation of the neural network hardware accelerator and integration into the Vivado block design.

Phase 5. Testing and benchmarking: Execution of inference on the FPGA and comparison against the hls4ml-generated core running on the ARM. Collection of performance metrics and verification of the obtained results.

Phase 6. Thesis report writing: Documentation of the development process, analysis of the obtained results, and final writing of the Bachelor's Thesis document.

Conclusions and Future Work

The main objective of this work was to design and evaluate a heterogeneous system on the Digilent Zybo board, using the ARM processor to control the application and the FPGA to accelerate a computationally intensive task. After completing the development process, it can be stated that this objective has been achieved. The final system is able to send an image from the ARM, transfer it to the programmable logic through AXI DMA, execute a convolutional model generated with hls4ml, and retrieve the output logits in order to obtain the predicted class.

The path toward this final version was not straightforward. The main difficulty was not only training a CIFAR-10 model, but making that model actually fit within a Zynq-7010. Many configurations that seemed reasonable in terms of parameter count became unfeasible after implementation in Vivado, especially due to LUT usage and slice packing issues. This part of the work made it clear that, on a small FPGA, discussing only the model size is not enough: the way operations are synthesized and placed on the device is just as important.

Achievement of the objectives

The first part of the project made it possible to bring up the platform and understand the complete Vivado/Vitis workflow. Bare-metal execution on the ARM was validated, the communication between the Processing System and the Programmable Logic was configured, and AXI-Lite and AXI DMA were used. These steps were necessary to ensure that the artificial intelligence component was not left as an isolated IP block without real system-level integration.

The matrix multiplication developed in Phase 3 served as an intermediate milestone. It was a more controlled test case than the CNN, but still sufficient to verify that Vitis HLS could generate competitive hardware and that the ARM-DMA-FPGA data path worked correctly. In addition, the comparison with a VHDL implementation helped assess the use of HLS from a more informed perspective. The tool does not remove the need to understand the underlying hardware, but it does significantly reduce the effort required to reach a functional parallel architecture.

In Phase 4, a complete CIFAR-10 model was successfully generated and integrated. The accuracy of the model had to be balanced against the resources available on the Zybo. The final version is not a large network, nor is it intended

to compete with modern computer vision models, but it does fulfill the purpose of this work: demonstrating AI inference running on dedicated hardware inside a resource-constrained FPGA.

Finally, the evaluation phase made it possible to measure the benefit of the acceleration. FPGA inference achieved an average execution time close to 2 ms, compared with approximately 171 ms for the hls4ml-generated C++ core running on the ARM. This represents an approximate speedup of 85 times. Beyond the exact numerical value, the important point is that the acceleration was measured on the real board and includes the communication cost with the IP core.

Lessons learned

One of the clearest conclusions of the project is that HLS simplifies development, but does not make it automatic. Directives, data types, and the reuse factor have a direct impact on the final resource usage. On several occasions, it was necessary to go back to the model, modify its architecture, or adjust the quantization because the design did not fit on the FPGA. In this regard, working with hls4ml was particularly interesting: it brings the machine learning workflow much closer to hardware design, but it still requires careful analysis of the synthesis, utilization, and timing reports.

It was also important to distinguish between the different validation levels. At first, when the execution on the board produced inconsistent results, it was tempting to assume that the model itself was faulty. However, part of the problem came from Vivado/Vitis cache issues and older versions of the IP that were still being used during integration.

Another practical lesson was the importance of keeping projects well organized. In a workflow involving Vivado, Vitis, Vitis HLS, trained models, generated IP cores, and auxiliary scripts, it is easy to end up with duplicated folders or old results mixed with newer versions. Separating the project into different phases, each with its own Vivado/Vitis project folder, makes the work easier to maintain and explain.

Limitations

The main limitation of the system is the board itself. The Zybo Legacy is a very useful platform for learning and prototyping, but the Zynq-7010 provides only a limited margin for implementing CNNs. The final model had to be small and tightly constrained in terms of resources. This directly affects the accuracy that can be expected and limits the complexity of the images that can be classified robustly.

Another limitation is that the benchmark focused on inference latency for specific images. A more complete evaluation would require running a larger test set and obtaining metrics such as overall accuracy, a confusion matrix, or per-class behavior. In this work, the priority was to close the full hardware-software workflow and measure the real acceleration achieved on the board.

Care must also be taken when interpreting the confidence value displayed in the terminal. The IP core returns logits, not probabilities. The confidence value is

computed in the application using an approximate softmax in order to make the results easier to read. Therefore, the predicted class and the logits are more relevant than the exact confidence percentage.

Future work

A clear line of future work would be to improve the model while preserving the resource constraints. Techniques such as weight pruning, quantization-aware training, depthwise separable architectures, or a more systematic search for layer-specific word lengths could be explored. The goal would be to increase accuracy without exceeding the LUT and BRAM limits of the board (Han et al., 2016; Howard et al., 2017).

It would also be interesting to study DSP usage in more detail. In the final design, DSP utilization is low, while LUTs remain the most critical resource. Future work could focus on enforcing a more controlled mapping of multiplications to DSP blocks, adjusting data widths to favor that mapping, and comparing different HLS strategies. It will not always be possible to move logic into DSP blocks, but this aspect deserves a deeper exploration.

Another natural improvement would be to automate the generation flow. At present, the process involves training the model, exporting it to hls4ml, generating the IP, integrating it into Vivado, exporting the XSA, and rebuilding the Vitis platform. Part of this flow could be unified through scripts that clean older versions, regenerate the IP, and prepare a final reproducible folder. This would reduce cache- and path-related errors, which were a real source of problems during the project.

From an evaluation perspective, the next step would be to test many more images instead of only individual cases. An application capable of reading images from an SD card, or even from a camera, would make it possible to evaluate the system in a way that more closely resembles a real use case. It would also be interesting to measure energy consumption, since one of the reasons for using FPGAs in Edge AI is not only latency, but also efficiency (Sze et al., 2017).

Finally, the FPGA accelerator could be compared against a more optimized ARM implementation, for example using NEON or libraries designed for microcontrollers and embedded processors (Lai et al., 2018). This would make it possible to better position the result against a software path specifically optimized for CPU execution. Even so, the work carried out already demonstrates the main idea: when the computation is adapted to the hardware and properly integrated into the system, the FPGA can significantly reduce inference latency even on a resource-constrained board.

Bibliografía

- AMD. Vitis high-level synthesis user guide (ug1399). 2023. Disponible en <https://docs.amd.com/r/2023.1-English/ug1399-vitis-hls/Introduction-to-Vitis-HLS> (último acceso, May, 2026).
- AMD. Axi dma logicore ip product guide (pg021) - gather mode. 2025a. Disponible en https://docs.amd.com/r/en-US/pg021_axi_dma/Scatter/Gather-Mode (último acceso, Mar, 2026).
- AMD. Axi dma logicore ip product guide (pg021) - introduction. 2025b. Disponible en https://docs.amd.com/r/en-US/pg021_axi_dma/Introduction (último acceso, May, 2026).
- AMD. C++ arbitrary precision fixed-point types. 2026a. Disponible en <https://docs.amd.com/r/en-US/ug1399-vitis-hls/C-Arbitrary-Precision-Fixed-Point-Types> (último acceso, May, 2026).
- AMD. Zynq 7000 soc technical reference manual (ug585). 2026b. Disponible en <https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/Introduction?tocId=Hf6C70o5ABvv2hkWRoihQ> (último acceso, Abr, 2026).
- DIGILENT. Digilent zybo (legacy). 2024. Disponible en <https://digilent.com/reference/programmable-logic/zybo/start> (último acceso, Mar, 2026).
- DIGILENT. Vivado board files for digilent fpga boards. 2025. Disponible en <https://github.com/Digilent/vivado-boards> (último acceso, Feb, 2026).
- FAHIM, F., HAWKS, B., HERWIG, C., HIRSCHAUER, J., JINDARIANI, S., TRAN, N., CARLONI, L. P., GUGLIELMO, G. D., HARRIS, P., KRUPA, J., RANKIN, D., VALENTIN, M. B., HESTER, J., LUO, Y., MAMISH, J., ORGRENCI-MEMIK, S., AARRESTAD, T., JAVED, H., LONCAR, V., PIERINI, M., POL, A. A., SUMMERS, S., DUARTE, J., HAUCK, S., HSU, S.-C., NGADIUBA, J., LIU, M., HOANG, D., KREINAR, E. y WU, Z. hls4ml: An open-source codesign workflow to empower scientific low-power machine learning devices. 2021. Disponible en <https://arxiv.org/abs/2103.05579> (último acceso, Feb, 2026).
- FAST MACHINE LEARNING LAB. hls4ml documentation: Software details. 2026. Disponible en <https://fastmachinelearning.org/hls4ml/details.html> (último acceso, May, 2026).

- FASTML TEAM. fastmachinelearning/hls4ml. 2025. Disponible en <https://github.com/fastmachinelearning/hls4ml> (último acceso, Abr, 2026).
- FCASPE. Bnn-pynq-zybo: Qnns on pynq with zybo-z7 compatibility. 2020. Disponible en <https://github.com/fcaspe/BNN-PYNQ-ZYBO> (último acceso, May, 2026).
- GUO, C., PLEISS, G., SUN, Y. y WEINBERGER, K. Q. On calibration of modern neural networks. En *Proceedings of the 34th International Conference on Machine Learning* (editado por D. Precup y Y. W. Teh), vol. 70 de *Proceedings of Machine Learning Research*, páginas 1321–1330. PMLR, 2017.
- HAN, S., MAO, H. y DALLY, W. J. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *ICLR*, 2016.
- HENNESSY, J. L. y PATTERSON, D. A. A new golden age for computer architecture. *Commun. ACM*, vol. 62(2), página 48–60, 2019. ISSN 0001-0782. Disponible en <https://doi.org/10.1145/3282307> (último acceso, Mar, 2026).
- HOWARD, A. G., ZHU, M., CHEN, B., KALENICHENKO, D., WANG, W., WEYAND, T., ANDREETTO, M. y ADAM, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. 2017. Disponible en <https://arxiv.org/abs/1704.04861> (último acceso, May, 2026).
- JACOB, B., KLIGYS, S., CHEN, B., ZHU, M., TANG, M., HOWARD, A., ADAM, H. y KALENICHENKO, D. Quantization and training of neural networks for efficient integer-arithmetic-only inference. En *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, páginas 2704–2713. 2018.
- KERAS. Globalaveragepooling2d layer. 2026. Disponible en https://keras.io/api/layers/pooling_layers/global_average_pooling2d/ (último acceso, May, 2026).
- KRIZHEVSKY, A. Learning multiple layers of features from tiny images. 2009. Disponible en <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf> (último acceso, May, 2026).
- LAI, L., SUDA, N. y CHANDRA, V. Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus. 2018. Disponible en <https://arxiv.org/abs/1801.06601> (último acceso, May, 2026).
- LIN, M., CHEN, Q. y YAN, S. Network in network. 2014. Disponible en <https://arxiv.org/abs/1312.4400> (último acceso, May, 2026).
- NVIDIA. Nvdla open source project. 2026. Disponible en <https://nvdla.org/primer.html> (último acceso, May, 2026).
- PUTNAM, A., CAULFIELD, A., CHUNG, E., CHIOU, D., CONSTANTINIDES, K., DEMME, J., ESMAEILZADEH, H., FOWERS, J., GRAY, J., HASELMAN, M., HAUCK, S., HEIL, S., HORMATI, A., KIM, J.-Y., LANKA, S., PETERSON, E.,

- SMITH, A., THONG, J., XIAO, P. Y., BURGER, D., LARUS, J., GOPAL, G. P. y POPE, S. A reconfigurable fabric for accelerating large-scale datacenter services. En *Proceeding of the 41st Annual International Symposium on Computer Architectecture (ISCA)*, páginas 13–24. IEEE Press, 2014. ISBN 978-1-4799-4394-4. Selected as an IEEE Micro TopPick.
- SZE, V., CHEN, Y.-H., YANG, T.-J. y EMER, J. Efficient processing of deep neural networks: A tutorial and survey. 2017. Disponible en <https://arxiv.org/abs/1703.09039> (último acceso, Abr, 2026).
- TENSORFLOW. formato hdf5. 2022. Disponible en https://www.tensorflow.org/tutorials/keras/save_and_load?hl=es-419#hdf5_format (último acceso, Abr, 2026).
- UMUROGLU, Y., FRASER, N. J., GAMBARDILLA, G., BLOTT, M., LEONG, P., JAHRE, M. y VISSERS, K. Finn: A framework for fast, scalable binarized neural network inference. En *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, FPGA '17, páginas 65–74. ACM, 2017.
- WU, R., GUO, X., DU, J. y LI, J. Accelerating neural network inference on fpga-based platforms—a survey. *Electronics*, vol. 10(9), 2021. ISSN 2079-9292. Disponible en <https://www.mdpi.com/2079-9292/10/9/1025> (último acceso, Mar, 2026).
- XILINX. Zynq-7000 soc data sheet: Overview. 2018. Disponible en <https://docs.amd.com/v/u/en-US/ds190-Zynq-7000-Overview> (último acceso, Mar, 2026).
- ZHOU, Z., CHEN, X., LI, E., ZENG, L., LUO, K. y ZHANG, J. Edge intelligence: Paving the last mile of artificial intelligence with edge computing. 2019. Disponible en <https://arxiv.org/abs/1905.10083> (último acceso, Feb, 2026).

